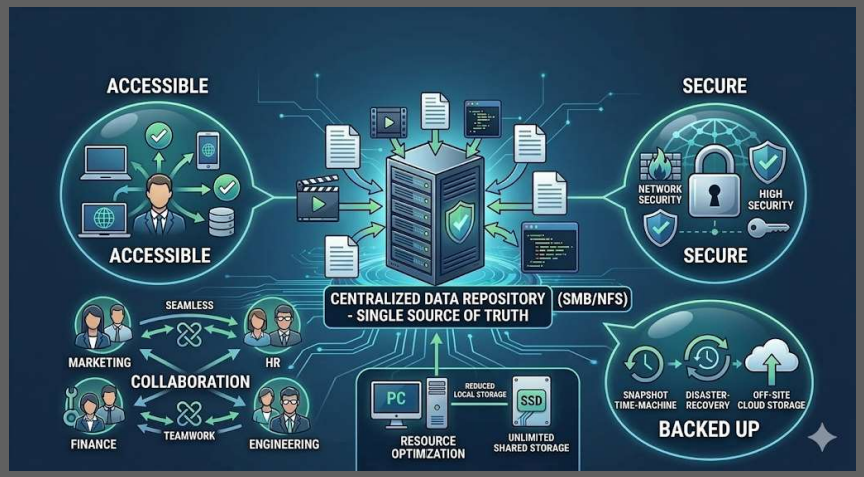
FILE SHARING ADMINISTRATION

INTRODUCTION TO FILE SHARING

- What is File Sharing?
  - The practice of distributing or providing access to digital media (documents, multimedia, programs) across a network.
  - A core responsibility of a System Administrator is to ensure files are accessible, secure, and backed up.
- Why is it important in an Enterprise?
  - Centralization: Prevents data silos and ensures a single "source of truth" for documents.
  - Collaboration: Enables multiple users to work on shared projects simultaneously.
  - Resource Optimization: Reduces the need for large local storage on individual workstations.



THE ROLE OF THE SYSADMIN IN FILE SHARING

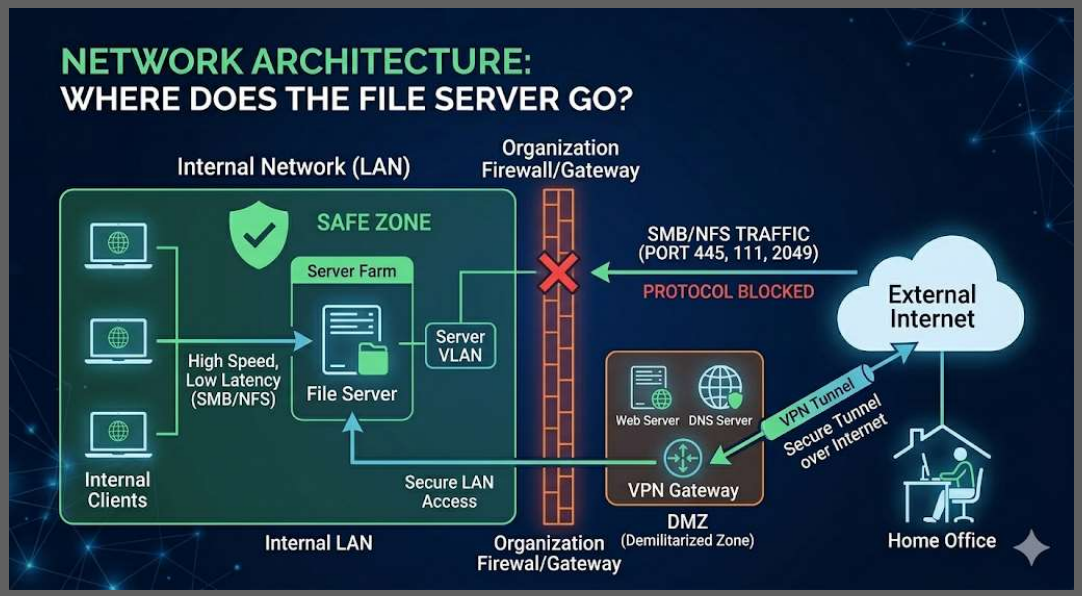
- Managing the Data Lifecycle
  - **Provisioning:** Setting up the server, allocating storage space (e.g., configuring LVM, Windows file share), and creating the shared directories.
  - **Access Management:** Creating users/groups, integrating with directories like Active Directory, and strictly enforcing permissions (ACLs).
  - **Maintenance & Monitoring:** Monitoring disk space (preventing 100% full drives), checking logs for unauthorized access, and applying security patches to SAMBA/NFS services.
  - **Disaster Recovery:** Implementing robust backup strategies (snapshots, off-site backups) to recover data in case of hardware failure or accidental deletion.

NETWORK ARCHITECTURE: WHERE DOES THE FILE SERVER GO?

- Implementing services in the organization's IT infrastructure requires strategic placement
- **The Internal Network (LAN) / Server Farm**
  - File servers handle massive amounts of internal data and require high-speed, low-latency connections
  - They belong deep inside the secure Internal Network or a dedicated Server VLAN
- **The DMZ Rule: Never Expose SMB/NFS**
  - A file server must NEVER be placed in the Demilitarized Zone (DMZ) or exposed directly to the public internet
  - Protocols like SMB and NFS were designed for local networks; exposing them directly leads to severe security breaches
- **How do Remote Workers Access Files?**
  - If employees are at home, they should not connect directly to the file server over the internet
  - Instead, they use a Virtual Private Network (VPN) to securely tunnel into the Internal LAN, and *then* access the file server as if they were in the office

NETWORK ARCHITECTURE: WHERE DOES THE FILE SERVER GO?

- **Safe Zone:**
  - The File Server is clearly placed deep within the Internal Network (LAN) or Server VLAN, protected by the Organization Firewall/Gateway
  - Internal clients connect with high speed and low latency
- **DMZ Rule (Blocked):**
  - Directly to the left of the firewall, any attempt from the **External Internet** (Public Internet/Rogue Actor) to connect direc%tly using **SMB/NFS Protocols (Ports 445, 111, 2049)** is visually blocked by a red 'X' and labeled **"PROTOCOL BLOCKED"**
- **Remote Access (VPN):**
  - A remote worker (Home Office/Public User) on the internet connects securely via a **"Secure VPN Tunnel"** to the **VPN Gateway**.
  - Once authenticated, they are visually placed *inside* the secure LAN to access the File Server, as if they were in the office



FILE SHARING PROTOCOLS: SMB/CIFS VS. NFS

- Server Message Block (SMB) / Common Internet File System (CIFS):
  - Primarily used in Windows environments.
  - Supports complex authentication (Active Directory integration) and file locking.
- Network File System (NFS):
  - The standard protocol for Unix/Linux environments.
  - Focuses on high performance and seamless mounting of remote directories as if they were local.

IMPLEMENTING FILE SHARING: SAMBA (SMB/CIFS)

- What is SAMBA?
  - An open-source implementation of the SMB networking protocol.
  - Allows a Linux server (like Ubuntu) to share files and printers with Windows clients.
- Key Features:
  - File and Print Services.
  - Active Directory Integration (can act as a Domain Controller or join a domain).
  - Granular access control using Windows-style ACLs (Access Control Lists).

SAMBA CONFIGURATION ON UBUNTU

- Installation:

```
sudo apt update
sudo apt install samba
```

- Configuration File: `/etc/samba/smb.conf`
- Basic Share Example:

```
[SharedDocs]
path = /srv/samba/shared
read only = no
browsable = yes
valid users = @staff
```

IMPLEMENTING FILE SHARING: NFS

- What is NFS?
  - Allows a client system to access files over a network much like local storage is accessed.
  - Ideal for server-to-server file sharing within a data center (e.g., sharing web server assets across multiple nodes).
- Key Components:
  - NFS Server: Hosts the shared directories.
  - NFS Client: Mounts the shared directories.
  - RPC (Remote Procedure Call): Used for communication between client and server.

NFS CONFIGURATION ON UBUNTU

- Server Installation:
  - `sudo apt update`
  - `sudo apt install nfs-kernel-server`
- Configuration File: `/etc/exports`
- Export Example:
  - `/srv/nfs/shared  192.168.1.0/24(rw,sync,no_subtree_check)`
- Explanation:
  - Shares the directory `/srv/nfs/shared`.
  - Restricts access to the `192.168.1.0/24` subnet.
  - Grants read/write (`rw`) access.

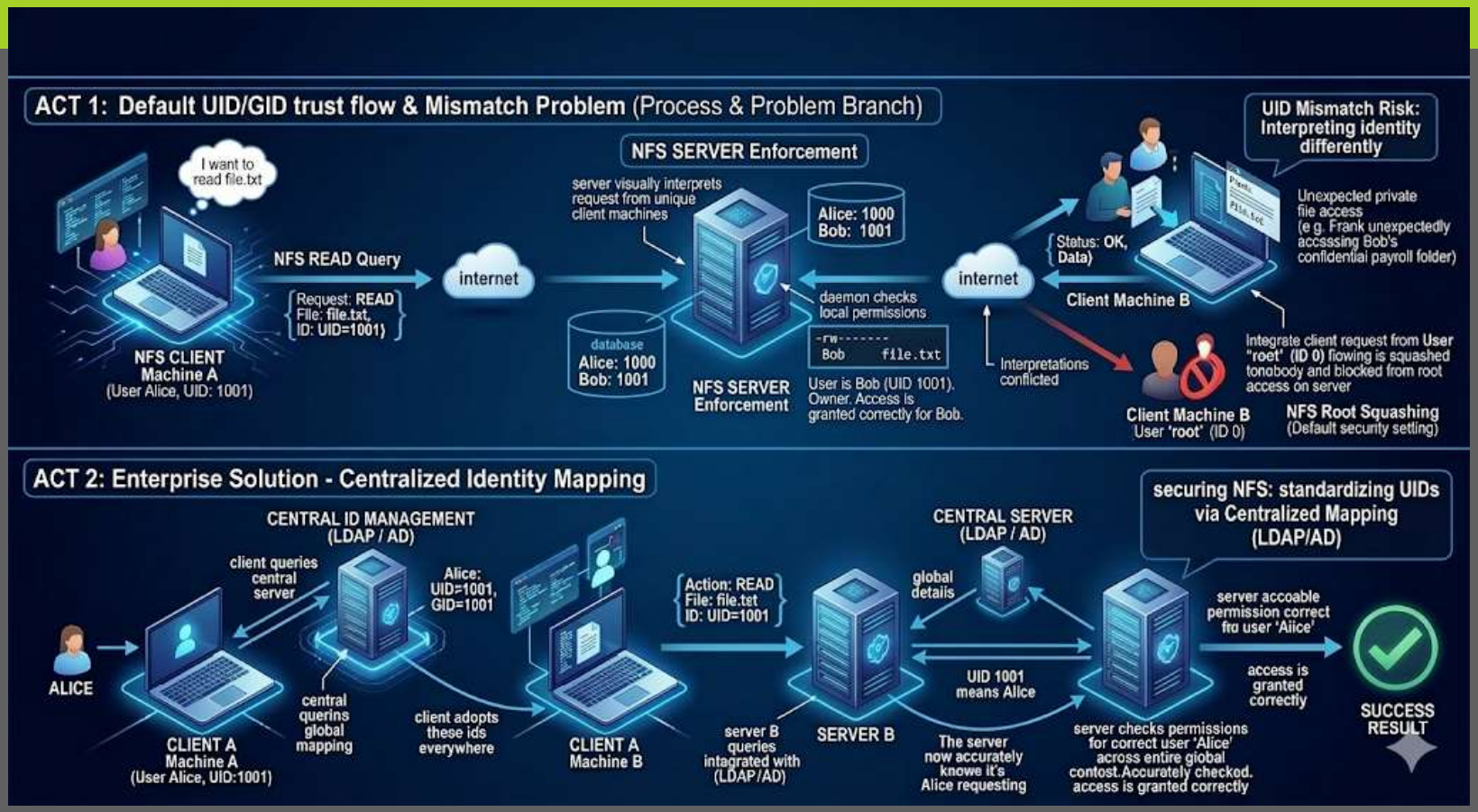
FILE PERMISSION SECURITY: THE BASICS

- The Importance of File Security in Sharing
  - Sharing a folder over the network exposes it. If permissions are too loose, anyone on the network can read, modify, or delete critical data.
  - It is the SysAdmin's job to enforce the Principle of Least Privilege: Users should only have the minimum access necessary to do their jobs.
  - Risks of Poor Permissions: Data breaches, ransomware spread, and accidental deletion of corporate records.
- Standard Permissions vs. ACLs:
  - Standard Linux (UGO): User, Group, Others (e.g., `chmod 755`). Too rigid for complex enterprise needs.
  - Access Control Lists (ACLs): Allows you to grant specific permissions to multiple different users and groups on a single file or directory.
- Using ACLs in Linux (`setfacl`):
  - Granting user 'alice' read/write access: `setfacl -m u:alice:rw /srv/samba/shared`
  - Checking permissions: `getfacl /srv/samba/shared`

HOW NFS HANDLES PERMISSIONS (CLIENT OR SERVER?)

- The Short Answer: Both, but the Server has the final say.
- The "UID/GID Trust" Problem:
  - By default, NFSv3 and NFSv4 do not authenticate the *user*, they trust the *Client Machine*.
  - When a user on the Client tries to read a file, the Client sends their User ID (UID) and Group ID (GID) to the Server. (e.g., "I am UID 1001").
- The Server's Role (Enforcement):
  - The Server receives "UID 1001 wants to read file.txt".
  - The Server looks at the local Linux file permissions for `file.txt`.
  - If the file allows read access to UID 1001, the Server grants it. If not, it denies it.
- The Catch (UID Mismatch):
  - If "Alice" is UID 1001 on the Client, but "Bob" is UID 1001 on the Server, Alice will accidentally have access to all of Bob's files on the NFS share!
- The Solution:
  - Centralized Identity Management (like LDAP or Active Directory) ensures that a user has the exact same UID/GID across all machines in the network.

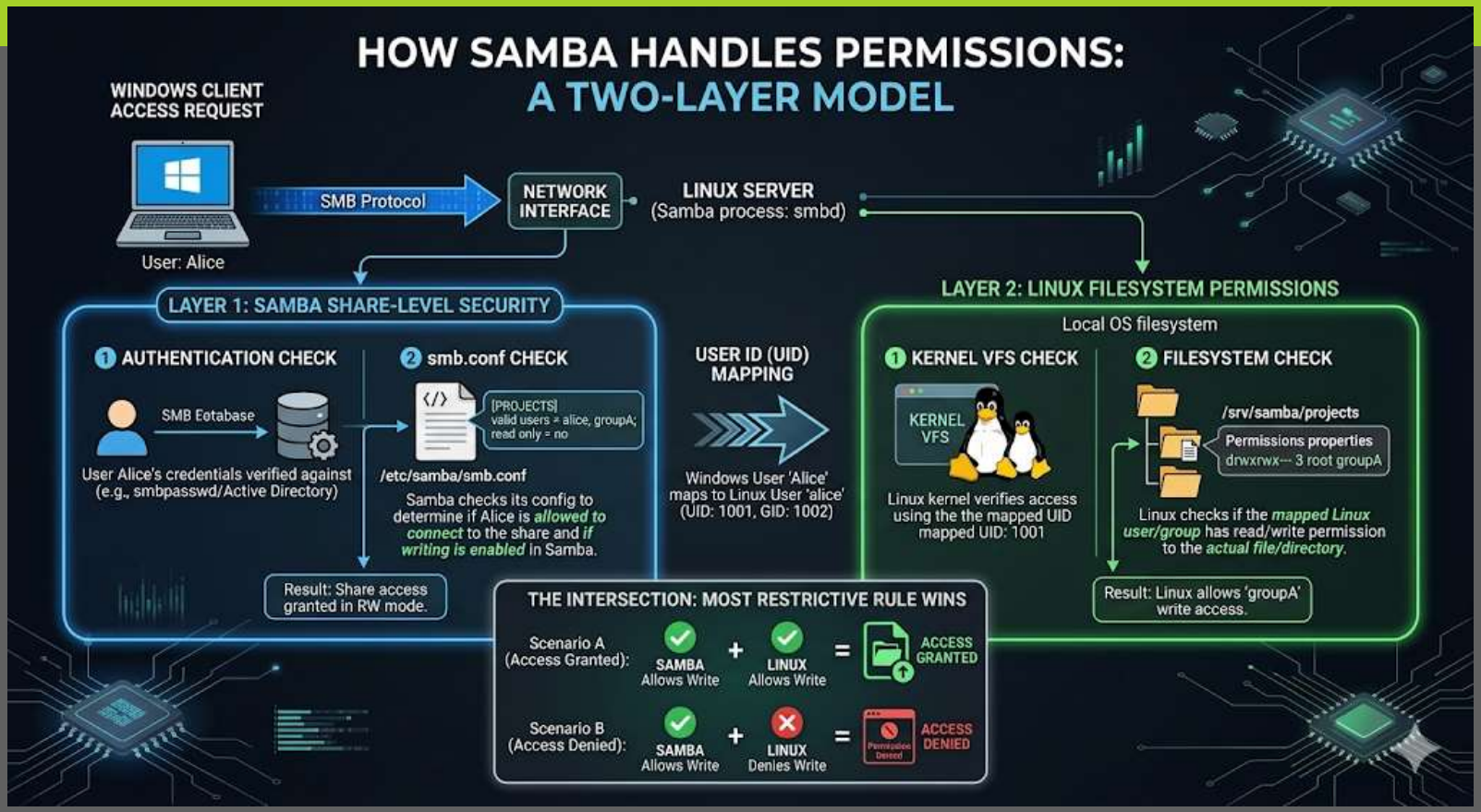
HOW NFS HANDLES PERMISSIONS



HOW SAMBA HANDLES PERMISSIONS (SHARE VS. FILESYSTEM)

- Two Layers of Security:
  - SAMBA uses a two-gate system. A user must pass BOTH gates to access a file.
  - Gate 1: Share Permissions (Defined in `/etc/samba/smb.conf` via `read only`, `valid users`, etc.).
  - Gate 2: Linux Filesystem Permissions (Defined by `chmod`, `chown`, and `setfacl` on the server directory itself).
- The "Most Restrictive" Rule:
  - The most restrictive permission between the Share and the Filesystem always wins.
  - Example: If `smb.conf` allows writing (`read only = no`), but the underlying Linux directory is read-only for that user, the write attempt will be DENIED.
- Mapping Users (Windows to Linux):
  - When a Windows user authenticates, SAMBA maps their credentials to a valid Linux user account (UID) on the server to check the Gate 2 filesystem permissions.
- Permission Forcing (`smb.conf`):
  - Sysadmins can use `force user = nobody` or `force group = staff` in `smb.conf`. This forces all actions in that share to be executed as that specific Linux identity, simplifying permissions for public folders.

HOW SAMBA HANDLES PERMISSIONS



SECURITY AND BEST PRACTICES

- NFS Root Squashing:
  - By default, if the 'root' user (UID 0) on the Client accesses an NFS share, the Server "squashes" them into an anonymous, unprivileged user (`nobody`).
  - This prevents a compromised client machine from having root-level control over the server's shared files.
- Access Control:
  - Implement the Principle of Least Privilege. Only grant access to users who absolutely need it.
  - Use robust authentication mechanisms.
- Encryption:
  - SMBv3 supports end-to-end encryption.
  - NFSv4 can utilize Kerberos for securing traffic.
- Monitoring and Auditing:
  - Regularly review access logs (e.g., `/var/log/samba/`) to detect unauthorized activity.
  - Monitor disk space usage and implement quotas if necessary.

FILE SHARING ADMINISTRATION